AeroCast AI

Predictive Airport Capacity & Delay-Risk Copilot

Forecast hourly runway Arrival Acceptance Rate and severe-delay risk 3-12 hours ahead of severe weather, so controllers act early instead of reactively.

Challenge #5 | Autonomous SG 2026 · AI & Aerospace (Airbus) Track Build spec · 4-hour hackathon · v1

In one sentence. A reproducible pipeline and a Streamlit control-room dashboard that ingests the provided flight+weather history for Atlanta (ATL) and Dallas-Fort Worth (DFW), enriches it with the one critical field the file is missing (wind direction), and predicts two things 3-12 hours ahead: the hourly **Arrival Acceptance Rate (AAR)** an airport can sustain, and which specific flights face **severe delay (> 45 min)** - each prediction carrying a confidence band, the weather drivers behind it, and a plain-English brief. The controller decides; the model advises.

This document is the lineage translation of the team's existing **PortSync AI** plan (a maritime terminal decision-support system) into the aviation Challenge #5. The architecture, the LLM / non-LLM split, and the "prove it against a baseline" discipline carry over almost unchanged; only the domain and data change. Every module below names its PortSync ancestor.

1 The ideal deliverable — what to aim for

Aim for a **working, reproducible, baseline-beating MVP with one honest failure case** — not a broad half-built system. The judges reward restraint and evidence over polish. Concretely, at the demo the judges should see six things, each mapped to a "Want to see" line in the brief:

#	What the judges see	Maps to rubric “Want to see”
1	A one-command run that ingests the CSV, pulls missing public weather, builds features, trains, and launches the app — no manual steps.	Automatic processing of public datasets
2	A forecast of hourly AAR for the next 3-12 h with a confidence band, overlaid on the typical/historical AAR for that hour.	Predictive performance (AAR, RMSE)
3	A capacity-reduction alert that fires with lead time when predicted AAR drops below the airport's normal band.	Decision support for ATC
4	A ranked list of upcoming flights flagged severe-delay-risk (> 45 min) with probability and confidence.	Delay risk classification
5	The weather drivers behind each alert (crosswind, gust, precip, condition) + a plain-English ATC brief.	Dashboard & decision support; trust
6	A metrics panel: RMSE vs baselines, classifier precision/recall/PR-AUC, calibration, and one shown failure case.	Honest evaluation

Definition of done

- **MVP (must ship in 4 h):** airport-generic pipeline demoed on ATL + DFW; AAR regressor and >45-min classifier both beating a named baseline; crosswind feature from enriched wind direction; Streamlit dashboard with confidence + one failure case; a `run.sh` that reproduces everything.
- **Stretch (only if MVP is solid):** conformal prediction intervals; leave-one-airport-out generalisation proof; live METAR/TAF pull; the LLM brief; SHAP attributions.

2 What it does and how it functions

The system runs as a **leakage-safe forecasting loop**. The core idea that keeps it honest: a prediction made at time *t* for a target hour *h* (where *h* is 3-12 h in the future) may use **only information knowable at *t*** — weather observations and forecasts already on record, the schedule, and climatology. It must never use the weather measured at hour *h* itself. That single rule defeats the brief's primary trap (“data leakage across time steps”).

End to end, one disruption forecast flows through eight stages:

- **Ingest & integrate** the flight+weather CSV; filter to the selected airports; align everything to an *airport-hour* grid using shared timestamps and station identifiers.
- **Enrich** each airport-hour with wind direction fetched from a public source (the CSV omits it), keyed by station + timestamp.
- **Engineer features** that exist at *t*: crosswind/tailwind vs runway heading, lagged weather at lead 3-12 h, gusts, precipitation, condition code, hour-of-day and seasonal terms, recent rolling throughput.
- **Build targets**: hourly AAR proxy per airport, and a per-flight binary severe-delay (> 45 min) label.
- **Predict**: a regressor forecasts AAR (with intervals); a classifier scores each upcoming flight's delay risk (calibrated).
- **Evaluate** against persistence and climatology baselines on a strict temporal hold-out; report RMSE, precision/recall, PR-AUC, calibration, per-airport and volume-normalised.
- **Explain**: convert the verified numbers into feature attributions, a confidence rating, and a short natural-language brief; flag low-confidence cases for human review.
- **Serve**: the dashboard presents forecasts, alerts, flagged flights, drivers and the brief to the controller, who decides whether to act.

Stages 1-6 are deterministic ML / statistics / rules (the parts that must be trustworthy). A language model appears only at stage 7, and only to *describe* verified outputs — it never computes a number or triggers an action. This is the exact split the rubric calls “technically sound”.

3 Modular architecture — what each module does

Nine modules, each a single Python file with one responsibility, wired by a thin orchestrator. Build and test them independently; integrate last.

Module (file)	What it does	PortSync ancestor
1. Ingest & Integration `ingest.py`	Loads the CSV, filters to target airports (ATL/DFW, but accepts any ICAO), parses dates/times into UTC-local hour buckets, and emits a clean **airport-hour** weather table plus a flight-level table. Handles the multi-source merge on timestamp + station id.	Terminal State Engine (clean, timestamp, validate, combine)
2. Weather Enrichment `enrich.py`	Fetches the **wind-direction** field the CSV lacks from Meteostat (same source as the existing columns) for KATL/KDFW by station + hour, caches it to parquet, and joins it on. This is the “automatic public-dataset processing” deliverable. Falls back gracefully (flagged) if offline.	External signals ingestion

Module (file)	What it does	PortSync ancestor
3. Feature Engineering `features.py`	Computes crosswind/tailwind/headwind vs runway heading; builds **lagged** weather at leads 3-12 h; adds gust, precip, snow, pressure, condition code, hour-of-day, day-of-week, month, and rolling recent throughput. Enforces the leakage rule (no same-hour target weather).	State Engine + Forecasting inputs
4. Target Builder `targets.py`	Derives the **AAR proxy** (completed arrivals per airport-hour, optionally the demand-normalised completion ratio) and the per-flight **severe-delay label** (ARR_DELAY > 45). Single source of truth for labels.	Predicted variables
5. Models `model_aar.py` / `model_risk.py`	XGBoost **regressor** for AAR (with quantile intervals) and a class-weighted **classifier** for delay risk (probability + calibration). One pooled, airport-generic model each.	Forecasting + Equipment-health risk
6. Baselines & Evaluation `evaluate.py`	Persistence and climatology baselines; RMSE + skill score; precision/recall/F1/PR-AUC; calibration (Brier); per-airport and volume-normalised reports; and automatic extraction of one **failure case**.	KPI & Carbon Scorer / honest evaluation
7. Explanation Copilot `copilot.py`	Turns a verified prediction into top feature drivers + a confidence tier, then an LLM one-paragraph ATC brief over structured JSON. Low-confidence -> “human review” flag. LLM explains only.	Operations Copilot + Human approval
8. Dashboard `app.py` (Streamlit)	The ATC control-room UI: pick airport + ‘now’; see the AAR forecast with band, capacity-reduction alerts, ranked risky flights, weather drivers, the brief, and the live metrics panel.	Streamlit dashboard
9. Orchestrator `run.sh` / `pipeline.py`	One entry point: ingest -> enrich -> features -> targets -> train -> evaluate -> launch. Makes the whole thing reproducible and ‘automatic’ end to end.	Closed decision loop

4 Data reality & the key decisions it forces

Findings from inspecting `Flight_Data_With_Weather_Final.csv` (148,273 rows, 2023-01 to 2025-07). These are decisions to make *before* coding — and stating them openly earns honest-evaluation points.

Finding in the data	Decision it forces
The file contains many airports (ATL, DFW, DEN, ORD, CLT, LAX...), not just two.	Filter to ATL + DFW for the demo, but key every step on airport code so the pipeline is generic (defeats the over-fitting trap).
Weather varies within each day (hourly granularity), matched to flight times.	Hourly AAR modelling is viable; aggregate to an airport-hour grid.
No wind-direction column — only temp, prcp, wspd, wpgt, pres, snow, coco.	Crosswind is impossible from the file alone. Enrich with public wind direction (Module 2). This *is* the ‘directionality’ requirement.
Severe delay (> 45 min) is only ~9.6% of flights.	Imbalanced classification: use class weights and judge by PR-AUC / recall, not accuracy.
WEATHER_DELAY / NAS_DELAY are ~79% missing and are post-hoc cause tags.	Do not use them as features (leakage + missingness). Use ARR_DELAY for the label; use them only in analysis.
No published AAR exists in the data.	Use a stated **proxy**: completed arrivals per airport-hour (and/or completion ratio). Name it as an assumption and show its limits.
Clean 2.5-year span.	Temporal split: train 2023-2024, test 2025. Never shuffle across time.

5 Modelling specification

Task A — Hourly AAR forecast (regression, scored by RMSE)

- **Target:** AAR proxy = count of completed arrivals at airport *a* in clock hour *h* (refinement: completion ratio = arrived / scheduled, which normalises demand).
- **Horizon:** predicted at *t* for *h* in [t+3h, t+12h].
- **Model:** XGBoost regressor; quantile objective (or residual bootstrap) for a prediction interval.
- **Metric:** RMSE and MAE on the 2025 hold-out; skill score = 1 - RMSE(model)/RMSE(baseline).

Task B — Severe-delay risk (classification, > 45 min)

- **Target:** per-flight binary, ARR_DELAY > 45 min.
- **Model:** class-weighted XGBoost classifier; probability calibration (isotonic/Platt).
- **Metric:** precision / recall / F1 at the operating threshold, PR-AUC, and a calibration curve (Brier). Report per airport and normalise by flight volume.

Runway-directionality features (the required physics)

With wind direction `wdir` (degrees) from Module 2 and the given runway headings (ATL = 80°, DFW = 180°), and theta = wdir - heading:

```
crosswind = wind_speed × |sin(theta)|

headwind = wind_speed × cos(theta)    (tailwind = max(0, -headwind))

compute the same for gust (wpgt) to get peak crosswind / peak tailwind
```

Baselines (non-negotiable — this is where honest-evaluation points live)

- **Persistence:** next hours look like the recent same-hour average.
- **Climatology:** historical mean AAR by hour-of-day and month.
- **No-weather model:** calendar features only — shows the marginal value of weather.
- The model must beat these, and the dashboard must show by how much.

6 Complete tech stack (4-hour-realistic)

Layer	Tool	Why this, for a 4-hour build
Language	Python 3.11	One language end to end; everyone on the team knows it.
Data wrangling	pandas, numpy	Fast CSV load, grouping to airport-hour, feature frames.
Weather enrichment	meteostat (+ requests fallback)	One call returns hourly wind direction for KATL/KDFW; same source as the file.
Modelling	scikit-learn, xgboost	Strong tabular baselines in minutes; feature importance for free.
Uncertainty	xgboost quantile / MAPIE (stretch)	Prediction intervals -> the ‘visible uncertainty’ the rubric rewards.
Evaluation	scikit-learn.metrics	RMSE, PR-AUC, calibration out of the box.
Dashboard	Streamlit + Plotly	A usable control-room UI in ~1 hour; no front-end code.
Explanation	Claude API (haiku/sonnet) over JSON; template fallback	Plain-English ATC brief; offline-safe fallback so the demo never breaks.

Layer	Tool	Why this, for a 4-hour build
Persistence	parquet + joblib	Cache enriched weather and trained models; no database needed.
Orchestration	run.sh / Makefile	One command = reproducible + ‘automatic processing’ tick.
Env	requirements.txt (or uv)	Reproducible setup; pin versions.

7 The 4-hour development pipeline

Timeboxed, with a 3-4 person parallel split. The rule: an ugly end-to-end system beats a beautiful half. Get a dumb version running by the 1.5-hour mark, then improve.

Time	Everyone / integration	Parallel tracks
0:00-0:20	Setup: repo, env, load CSV, confirm columns, agree target + split.	All together.
0:20-1:30	First dumb end-to-end skeleton.	A: ingest + airport-hour grid. B: baselines + first XGBoost on a few features. C: Streamlit shell with dummy data.
1:30-2:30	Wire real features into models; first honest metrics.	A: crosswind + wind-dir enrichment. B: train AAR + classifier, metrics vs baselines. C: dashboard reads real outputs.
2:30-3:15	Integrate; add trust layer.	B: calibration + intervals. C: risk flags, drivers, LLM brief. A: find the failure case.
3:15-3:45	Freeze features; polish evidence.	Metrics table, calibration plot, per-airport + normalised view. Stop adding features.
3:45-4:00	Rehearse 3-min demo; submit early.	Time discipline is 5% — submit with minutes to spare.

- **Cut line:** if behind at 2:30, drop the LLM brief and stretch items — keep baselines, metrics, crosswind, and the dashboard. Those carry the score.

8 How this wins the rubric

Criterion (weight)	What in this build earns it
Technical soundness (25%)	End-to-end run; ML for prediction, rules/trig for crosswind, solver-free stats for baselines, LLM only to explain — the exact split the rubric calls strong. Tested beyond one scripted case.
Problem realism (20%)	Named user (ATC duty manager), real status quo (reactive capacity cuts, airborne holding, fuel burn), narrow scoped workflow.
Honest evaluation (20%)	Three baselines, temporal hold-out, RMSE + PR-AUC + calibration, one shown failure case, explicit AAR-proxy assumption.
Trust & traceability (20%)	Every alert shows its weather drivers + confidence; low-confidence -> human review; the controller decides.
Impact (10%)	Quantified lift over baseline (RMSE skill score) and lead-time framing vs today's reactive cuts.
Time discipline (5%)	Reproducible one-command run; rehearsed 3-min demo; early submission.

9 Assumptions & risks (state these out loud)

- **AAR is a proxy.** Completed arrivals/hour conflates demand with capacity; in low-demand hours it understates true capacity. Mitigation: model reductions relative to the typical band, or use the completion ratio.

- **Wind direction is fetched, not given.** Needs network at build time; cache it early and ship a flagged magnitude-only fallback so the demo still runs offline.
- **Generalisation claim is scoped.** The pipeline is airport-generic, but we only validate on ATL + DFW (the two with given headings). Say so; offer leave-one-airport-out as the proof if time allows.
- **‘Forecast’ here means lagged observations** (a persistence-style forecast built from logs available before the event), not a numerical weather prediction. This is leakage-safe and honest; real TAF ingestion is a stretch.

AeroCast AI is the aviation translation of the team's PortSync AI plan. Name is a placeholder — rename freely. This spec is scoped so the MVP is finishable in four hours and every hour of work maps to a scored criterion.